

Twyne - PRs review

Prepared by: Josselin Feist (josselin@seceureka.com)

April 13th, 2026

Prepared For: Twyne

Twyne - PRs review

Executive Summary	3
Overview	3
Security findings	5
Divergence between configured and effective LTV bounds	6
Summary	6
Detailed description	6
Recommendations	7
Status	7
Appendix A	8

Executive Summary

Timeline

- *March 9th: initial 1 week review (commit f15c596c)*
- *March 23th: half of a day review (commit 12a1ec2a)*
- *April 13th: review of the equivalence between the public repo ([0aa37b0](#)) and the reviewed commit (12a1ec2a) in [Appendix A](#)*

Overview

During the week of March 9th, we reviewed the recent changes made to the Twyne contracts.

No significant issues were identified. The review focused on identifying changes that could introduce new attack vectors or invalidate existing assumptions. Some of the risks were already known, including sandwiching collateral vault actions to modify available liquidity on the intermediate vault, and the impact of setting certain vault manager parameters to zero.

The review focused on the following pull requests:

PR 230, 239, 244, 252

These PRs allow the same assets to be used by multiple intermediate vaults and introduce a ramp mechanism for both the maximum LTV and the external liquidity buffer.

Two minor improvements could be considered:

- validate `isIntermediateVault` across the relevant setter functions to reduce the risk of misconfiguration.
- Consider introducing a separate function when `_rampDuration` is below a defined threshold (ex: an “immediate action” function). This would simplify access control verification for instantaneous parameter changes.

PR 231, 250, 246, 253

These PRs are related to the introduction of the dynamic liquidation LTV and improvement to the bad debt settlement.

We note that the current formulas make the definition of “dynamic liquidation LTV” increasingly difficult to interpret. In practices the system is evolving toward the usage of two LTVs:

- Liquidation LTV: the threshold used to determine whether the CV is liquidatable from Twyne’s perspective and the external condition (with buffer)
- Rebalancing LTV: derived from the same base logic, but incorporating an additional liquidity component from the intermediate vault

We recommend explicitly defining these two LTV concepts, particularly if the system continues to evolve. This would clarify both the intended behavior and the expected invariants.

Given the rebalancing is now dependent on a user-parameter while being permissionless, we also recommend documenting why rebalance does not have an account/vault status check, and carefully evaluate these choices as the system evolves.

Finally we would also recommend documenting that no donation-like function from non-borrower (to increase `totalAssetsDepositedOrReserved`) should be added to the contract without changing the bad debt settlement mechanism.

PR 234

This PR refactors the interest rate model and introduces a minimal interest component.

PR 241

This PR introduces additional access controls for pausing the contract.

PR 243

This PR fixes an issue that can occur when the contract attempts to release more assets than it holds. We suspect that this PR was needed (in part) to mitigate issue 001.

PR 217, 233, 240, 242, 235

These PRs relate primarily to minor changes in deployment scripts and documentation and were therefore not a primary focus of this review.

On March 23th, we reviewed PR259, and PR256 (fix for issue [001](#))

PR259 improves the liquidation flow by deferring the transfer of collaterals to the vault's owner at the end of the batch transaction, preventing the need for the liquidator to have collateral assets before the liquidation.

We noted a side effect of the PR: it is not possible anymore to liquidate the same vault multiple times within a batch. However this does not introduce security issues per our analysis.

Given the current complexity of the asset transfers and the continuous evolution of the codebase, we recommended improving the documentation of the flow of assets.

Security findings

ID	Title	Severity	Likelihood	Status
1	Divergence between configured and effective LTV bounds	<i>Informational</i>	<i>Low</i>	<i>Fixed</i>

Divergence between configured and effective LTV bounds

Severity: Informational

Likelihood: Low

Status: fixed

001

Summary

A discrepancy in how ramped values are considered in the LTV computation may introduce risks in future modifications of the codebase.

Detailed description

The chosen LTV is bounded by two parameters (from the vault manager):

- `externalLiqBuffers * buffer`
- `maxTwyneLTVs`

```
function _checkLiqLTV(uint _liqLTV) internal view {
    address __intermediateVault = address(intermediateVault);
    require(
        _getExtLiqLTV() *
        uint(twyneVaultManager.externalLiqBuffers(__intermediateVault))
        <= _liqLTV * MAXFACTOR
        && _liqLTV <=
        uint(twyneVaultManager.maxTwyneLTVs(__intermediateVault)),
        ValueOutOfRange()
    );
}
```

Figure 1: [CollateralVaultBase.sol#L118-L123](#)

Both `maxTwyneLTVs` and `externalLiqBuffers * buffer` may be modified after the chosen LTV has been set. From a high level, these parameters define lower and upper bounds for the chosen LTV.

When computing the collateral amount, the chosen LTV is bounded to be below the current `maxTwyneLTVs`:

```
userCollateral * MAXFACTOR * Math.min(twyneLiqLTV,
    twyneVaultManager.maxTwyneLTVs(address(intermediateVault)))
```

Figure 2: [CollateralVaultBase.sol#L447](#)

However, this check does not account for potential changes to the lower bound (`externalLiqLTV * buffer`).

In particular instead of using of $\min(\text{LiqLTV}, \text{upper})$, using the current system's parameter would lead to use $\min(\max(\text{lower}, \text{LiqLTV}), \text{upper})$.

Ignoring updates to the lower bound is conservative, as it favors liquidation over user protection when the user does not update their chosen LTV. However, this undocumented divergence could create inconsistencies as the system evolves.

For example, `_collateralScaledByLiqLTV1e8` divide the resulting collateral with an up to date version of the lower bound parameter:

```
uint adjExtLiqLTV =
uint(twyneVaultManager.externalLiqBuffers(address(intermediateVault))) *
_getExtLiqLTV();
    // When dynamic leg is selected: ceilDiv(adjExtLiqLTV * X,
adjExtLiqLTV) = X (exact, no rounding)
    // When chosen leg is selected: rounds up, which is conservative
(reserves more collateral)
    return Math.ceilDiv(_collateralScaledByLiqLTV1e8(true,
adjExtLiqLTV), adjExtLiqLTV);
```

Figure 1: [AaveV3CollateralVault.sol#L147](#)

No exploitability was identified in the current codebase using this vector.

Recommendations

Document the expected behavior and invariants associated with the bounds applied to the chosen LTV.

Status

Fixed with PR 260.

Appendix A

The following lists the differences between the private codebase we reviewed (12a1ec2a), and the public version (<https://github.com/0xTwyne/twyne-contracts-v1>, commit 0aa37b0):

```
# Root / Config:
- .env.example — addresses, chain, Tenderly keys
- .gitignore — ignore pattern change
- .gitmodules — submodule reordering
- foundry.lock — aave-v3-origin v3.5.0 → v3.6.0
- remappings.txt — trailing newline only

# Documentation:
- README.md — deployment instructions rewritten

# Tests:
- test/twyne/mainnet/euler/EulerOperators.t.sol
- test/twyne/mainnet/euler/EulerTestBase.t.sol

# Libraries:
- lib/forged-safe/src/BatchScript.sol — Safe API endpoint URLs
- lib/aave-v3-origin/** — dozens of files (v3.5 → v3.6 upgrade)
```

Additionally, some files are only present in the version we reviewed, but are not listed in this document as they are private.

None of the files that differ were directly in scope for the review.